

# Introduction to Machine Learning for Environmental Science

Stefan Siegart

This 2-day workshop introduces core concepts of machine learning including common algorithms, standard workflows and key software libraries. The focus will be on supervised machine learning for regression and classification using a range of methods such as random forests and deep convolutional neural networks. The material will be taught through a mix of theory and practical examples across different environmental sciences such as meteorology and ecology.

## Introduction

- Supervised ML
- Model training
- Examples, model evaluation

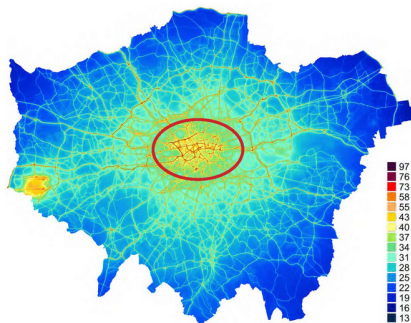
## Tree-based methods

- Decision trees
- Random Forest, XGBoost

## Neural Networks

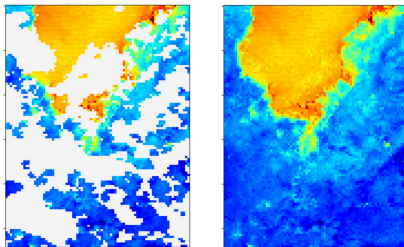
# Some examples

**Air quality prediction:** Predict high-resolution spatial pattern of next day PM10 concentration based on learned relationships between pollution, weather and traffic.



# Examples of ML applications

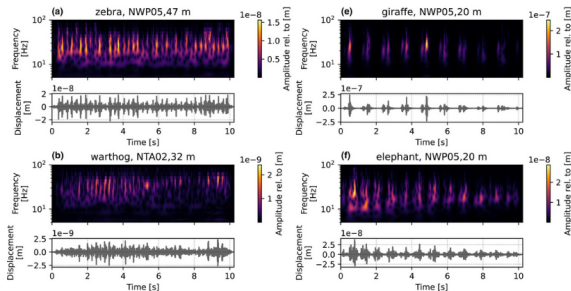
**Gap filling:** Fill in missing data due to clouds in satellite images based on learned spatial patterns.



<https://doi.org/10.3390/rs12233865>

# Examples of ML applications

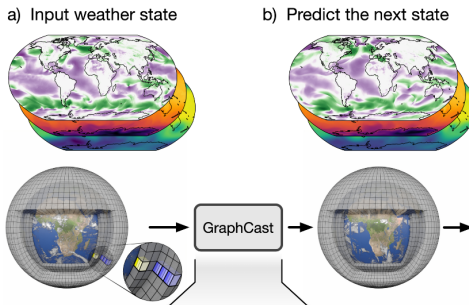
**Monitoring wildlife:** Learning patterns in seismic signals to detect and monitor animal species.



<https://doi.org/10.1111/2041-210X.70021>

# Examples of ML applications

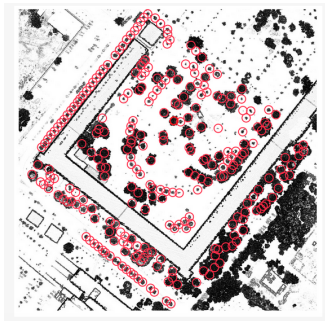
**Weather prediction:** Learning spatial, temporal and intervariable relationships between weather quantities to make skilful global weather predictions.



<https://doi.org/10.48550/arXiv.2212.12794>

# Examples of ML applications

**Image segmentation:** Detecting and annotating trees on urban maps based on learned image patterns.



<https://doi.org/10.3390/ijgi11040226>

# Examples of ML applications

What do these examples have in common?

- High-dimensional data (space, time, multivariable)
- Need to learn complex interactions between variables
- Data-rich but often limited mechanistic understanding
- Clear framing as an input-output relationship
- Focus on skilful prediction of observables



# Supervised machine learning: General setup

- We have an observable **target outcome** we wish to predict.
- The prediction can be based on a set of **input features**.
- For example
  - predict future temperature based on current temperature, pressure, wind
  - predict occurrence of wildfire based on measures of heat, humidity, wind
  - predict crop yield based on agricultural practice, soil nutrients, pests
- The goal of supervised ML is to **build a model** that predicts outcomes based on available features.
- A "good" ML model is one that produces **accurate predictions**.

# Regression and classification

- The type of target variable determines the type of ML problem.
- **Regression:** Predicting quantitative outcomes.
  - For example: temperature in degrees, crop yield in tonnes, number of sharks, rain amount in mm
- **Classification:** Predicting categorical outcomes.
  - For example: wildfire yes/no, temperature freezing/nonfreezing, precipitation type rain/snow/hail, air quality low/medium/high
- Regression vs classification determines various details of the ML model and analysis.

# WIP: Regression and classification

TODO: example outputs from a regression model and a classification model used later in the course to illustrate the distinction

# Function approximation theory

- A ML model is a **prediction rule** that translates one or more features  $x_1, x_2, \dots$  into an estimate  $\hat{y}$  of the target  $y$ .
- The prediction rule is calculated by a **mathematical function**

$$\hat{y} = f(x_1, x_2, \dots)$$

- The function  $f(\dots)$  usually includes **model parameters**  $\theta_1, \theta_2, \dots$  that control how inputs are translated to outputs.
- Defining the feature vector  $\mathbf{x} = (x_1, x_2, \dots)$  and parameter vector  $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots)$  a trainable ML model is often compactly written as

$$\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$$

# Training data

- Supervised ML works by "learning from examples".
- **Training data**  $S$  is a set of input-output examples

$$S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

where

- $\mathbf{x}_i = (x_{i,1}, x_{i,2}, \dots, x_{i,k})$  is the **feature vector** (containing  $k$  features) in the  $i$ -th example
- $y_i$  is the **target outcome** observed in the  $i$ -th example
- $n$  is the **sample size**
- The data set  $S$  is used to train our ML model  $f(\mathbf{x}; \theta)$  that can predict the outcome  $y^*$  for arbitrary feature vector  $\mathbf{x}^*$ .

# Loss function

- A loss function  $L(\hat{y}, y)$  is a function that quantifies **how different** the prediction  $\hat{y}$  is to the target outcome  $y$ .
- Loss functions are typically **negatively oriented**, where lower values indicate better predictions.
- The loss function must be appropriate for the ML task at hand (regression vs classification) and can usually be set when initialising the ML model.
- Otherwise the choice of loss function is **often ambiguous** and difficult to fully justify a particular choice.

# Loss function

- **Regression** (where  $\hat{y}$  and  $y$  are real numbers):
  - Squared-error loss  $L(\hat{y}, y) = (\hat{y} - y)^2$
  - Absolute-error loss  $L(\hat{y}, y) = |\hat{y} - y|$
- **Binary classification** (where  $y$  is 0 or 1 and  $\hat{y}$  is the probability for outcome "1"):
  - Cross-entropy loss  $L(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$
  - Quadratic loss  $L(\hat{y}, y) = (\hat{y} - y)^2$
- **Multivariate classification** (where  $y = (0, \dots, 1, \dots, 0)$  is a binary vector indicating which of  $k$  classes occurs and  $\hat{y}$  is a vector of  $k$  class probabilities)
  - Cross-entropy loss  $L(\hat{y}, y) = -\sum_{i=1}^k y_i \log \hat{y}_i$
  - Focal loss  $L(\hat{y}, y) = -\sum_{i=1}^k y_i (1 - \hat{y}_i)^\gamma \log \hat{y}_i$

# Empirical loss minimisation

- We can now **formalise** training of ML model  $f(\mathbf{x}; \theta)$  as
  - trying to find model parameters  $\theta$  such that
  - the model outputs  $\hat{y}_1, \dots, \hat{y}_n$  calculated from features  $\mathbf{x}_1, \dots, \mathbf{x}_n$
  - achieve minimal average loss when compared to the targets  $y_1, \dots, y_n$  in the training data set
  - using an appropriate loss function  $L(\hat{y}, y)$ .
- **The Empirical Loss Minimisation paradigm for ML:**

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n L[f(\mathbf{x}_i; \theta), y_i]$$

- The trained model with parameters  $\hat{\theta}$  can then be used to make predictions  $\hat{y}^*$  based on new feature vectors  $\mathbf{x}^*$

$$\hat{y}^* = f(\mathbf{x}^*; \hat{\theta})$$



## In summary ...

To train a ML model by empirical loss minimisation we need:

- a training data set  $S$  containing  $n$  examples of inputs and corresponding outcomes
- a model function  $f(\mathbf{x}; \theta)$  with trainable parameters  $\theta$  that translates inputs  $\mathbf{x}$  into an output  $\hat{y}$
- a loss function  $L(\hat{y}, y)$  that quantifies how close model output  $\hat{y}$  is to the target outcome  $y$
- a search algorithm to find parameters that minimise the loss

The combination of training data set, search algorithm, model function  $f(\mathbf{x}; \theta)$  and loss function  $L(\hat{y}, y)$  determine the ML approach and **these details should always be reported when documenting your ML model development.**

## Code example: Linear regression

- **Features**  $x_1, \dots, x_n$ : temperatures at different times  $t_1, \dots, t_n$
- **Targets**  $y_1, \dots, y_n$ : temperatures a short time  $\Delta$  later  
 $t_1 + \Delta, \dots, t_n + \Delta$
- **Model**: Linear function with intercept  $\theta_1$  and slope  $\theta_2$ , such that

$$\hat{y} = f(x; \theta) = \theta_1 + \theta_2 \cdot x$$

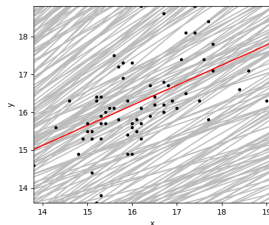
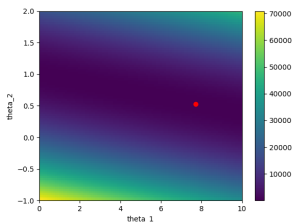
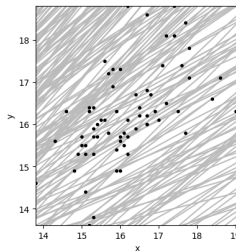
- **Loss**: Squared-error loss

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

- **Search**: Exhaustive search over a grid of parameter values  
(don't do this in practice!)

# Code example: Linear regression

```
In [129]: np.stack([x,y]).T
Out[129]:
array([[16. , 15.6],
       [15.9, 14.9],
       [15.9, 16.3],
       [15.2, 16.4],
       [13.8, 14.6],
       [16.7, 18.6],
       [15.3, 13.8],
       [16.1, 15.8],
       [15.1, 15.5],
       [16.7, 16.8],
       [14.9, 15.3],
       [15.3, 15.9],
       [15.1, 15.3]
```



```
In [164]: np.stack([x,y,y_hat]).T
Out[164]:
array([[16. , 15.6, 16.18 ],
       [15.9, 14.9, 16.127],
       [15.9, 16.3, 16.127],
       [15.2, 16.4, 15.756],
       [13.8, 14.6, 15.014],
       [16.7, 18.6, 16.551],
       [15.3, 13.8, 15.809],
       [16.1, 15.8, 16.233],
       [15.1, 15.5, 15.703],
       [16.7, 16.8, 16.551],
       [14.9, 15.3, 15.597],
       [15.3, 15.9, 15.809],
       [15.1, 15.3, 15.703],
       [15.1, 14.4, 15.703],
       [15.9, 15.4, 16.127],
       [16. , 15.7, 16.18 ],
       [15.1, 15.3, 15.862]
```

# Machine Learning in python with scikit-learn

<https://scikit-learn.org>

 [Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#) 1.7.2 (stable)

## scikit-learn

Machine Learning in Python

[Getting Started](#) [Release Highlights for 1.7](#)

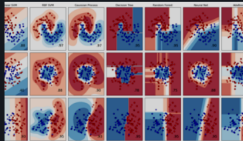
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

### Classification

Identifying which category an object belongs to.

**Applications:** Spam detection, image recognition.

**Algorithms:** [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)



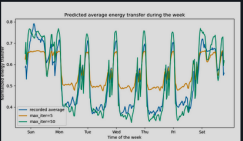
Examples

### Regression

Predicting a continuous-valued attribute associated with an object.

**Applications:** Drug response, stock prices.

**Algorithms:** [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)



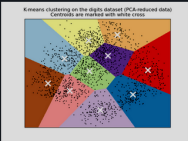
Examples

### Clustering

Automatic grouping of similar objects into sets.

**Applications:** Customer segmentation, grouping experiment outcomes.

**Algorithms:** [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



Examples

# Linear Regression in Scikit-Learn

```
from sklearn.linear_model import LinearRegression

# data wrangling:
# inputs: X - (n x k) array of n examples, k features
# targets: y - 1d array of length n

# initialise a linear regression model
model = LinearRegression()

# fit the model to data
model.fit(X, y)

# extract parameter estimates from fitted model
theta_hat = np.array([model.intercept_, model.coef_[0] ])

# calculate predictions in training data
y_hat = model.predict(X)

# calculate predictions on new data
x_new = np.linspace(15, 20, 100).reshape(-1,1)
y_new = model.predict(x_new)
```

# How good is my model?

- The model outputs  $\hat{y}_1, \dots, \hat{y}_n$  must be compared to the targets  $y_1, \dots, y_n$  using a suitable error metric.
- For example root mean squared error (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

- Our model has  $RMSE \approx 0.9$  – Is that "good"?
- Getting  $RMSE > 0$  means our model is not perfect, but how bad is it?
- To judge whether the model is useful we should compare it to a suitable benchmark.

# Benchmarking

- A benchmark model (or reference model) for performance evaluation is usually one or several of
  - An alternative (competing) model for the same target.
  - Climatology (constant mean):

$$\hat{y}_i^{(clim)} = \frac{1}{n} \sum_{i=1}^n y_i$$

- Persistence (last available observation of the target); here:

$$\hat{y}_i^{(pers)} = y(t_i - \Delta)$$

- Any other simple estimate of the target that could be calculated with reasonable effort given the same inputs as our model.
- In fact, linear regression itself is often a good reference to benchmark more complicated ML model.

# Skill scores

- Given a suitable error metric, we calculate
  - $S$ : the error of our model (averaged over a test data set)
  - $S_{ref}$ : the mean error of our chosen reference model
  - $S_{perf}$ : the mean error of a hypothetical perfect model that outputs  $\hat{y}_i = y_i$  each time; (usually  $S_{perf} = 0$ )
- The skill score of our model relative to the benchmark is then defined as

$$Skill = \frac{S_{ref} - S}{S_{ref} - S_{perf}}$$

- $Skill \leq 0$ : Our model is not better than the benchmark.
- $Skill \in (0, 1)$ : Our model improves over the benchmark.
- $Skill = 1$ : Our model is perfect.



# Skill scores

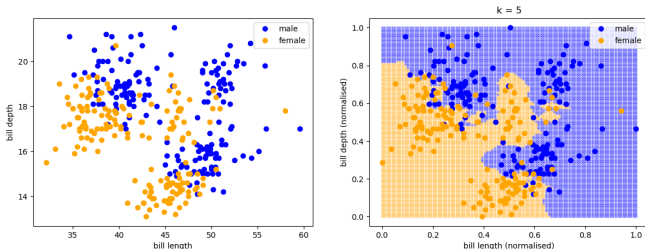
MORE HERE Skill scores of linear regression model

# Example: k-nearest-neighbor classification

- given an input  $\mathbf{x}^*$  find the  $k$  closest features  $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}$  in the training data, and the corresponding outcomes  $y^{(1)}, \dots, y^{(k)}$
- "closeness" between the feature vectors  $\mathbf{x}^{(i)}$  and  $\mathbf{x}^{(j)}$  is defined in terms of the Euclidean distance  $\sqrt{\sum_m (x_m^{(i)} - x_m^{(j)})^2}$
- predict  $\hat{y}^*$  as the majority vote over  $y^{(1)}, \dots, y^{(k)}$
- ties are resolved by picking the outcome class lowest value or alphabetical rank

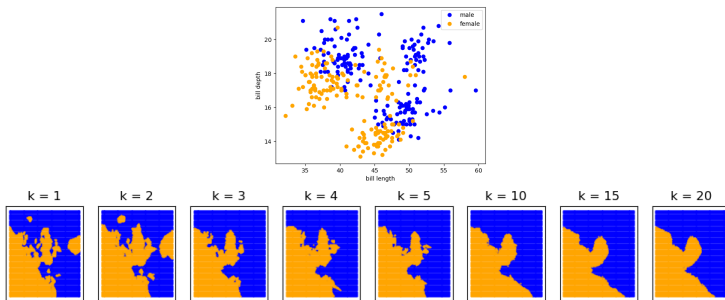
## Example: k-nearest-neighbor classification

- For illustration we use data from the "Palmer Penguin" data.
- Features are penguins' bill length ( $x_1$ ) and bill depth ( $x_2$ ) and outcome ( $y$ ) is the penguin's sex (male or female).
- The kNN classifier ( $k = 5$ ) separates the  $x_1/x_2$  plane into regions for "female" and "male", separated by a decision boundary.



# Example: k-nearest-neighbor classification

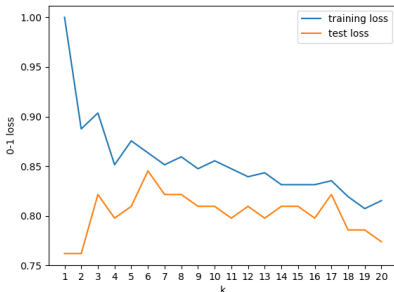
- The parameter  $k$  controls the "roughness" of the classifier.
- Larger values of  $k$  produce smoother decision boundaries.



- Is  $k$  a trainable parameter that can be selected by empirical loss minimisation?

## Example: k-nearest-neighbor classification

- Split the data randomly into training and test data set:
  - Only the training data set is used by the kNN classifier.
  - Training and test data are used separately to calculate the 0-1 loss (aka accuracy, aka proportion correct) of the classifier.



- The training accuracy is perfect for  $k = 1$  (Why?) and goes down as  $k$  increases.
- The test accuracy is best for  $k \approx 6$ .
- The neighborhood parameter  $k$  can be optimised by loss minimisation, but should use a different data set than the one used to define the kNN classifier.

# In-sample vs Out-of-sample error

- Our goal is to train a model that generalises well to new data.
- Loss on training data (in-sample error) is often too optimistic because the model "has seen" the data.
- Overfitting: A model achieves very low training loss by "regurgitating" the training data, including any noise and accidental patterns. Overfitted models generalise poorly to new data.
- Loss on previously unseen data (out-of sample error) is what we are really interested in.
- It is good practice to remove a fraction of data (e.g. 20%) during training and use only for model testing.

## Further reading

- Linear algebra for least squares regression.
- Alternative loss functions
- Multiple linear regression for multiple inputs.
- Logistic regression for classification.
- Radius-neighborhood classifier and other neighborhood methods.
- Neighborhood methods for missing data imputation.
- Uncertainty estimation: Resampling methods, bootstrapping.

## Introduction

- Supervised ML
- Model training
- Examples, model evaluation

## Tree-based methods

- Decision trees
- Random Forest, XGBoost

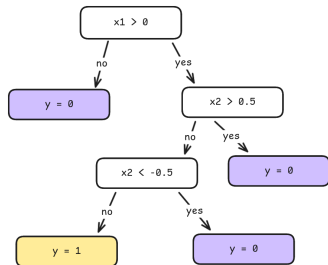
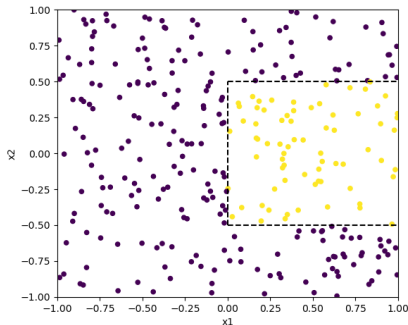
## Neural Networks



# Background

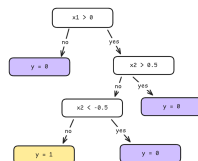
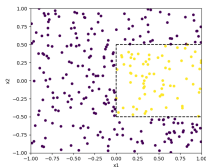
- Tree-based methods are simple and powerful function approximation methods.
- During training, the feature space is split up into rectangular regions.
- The model's prediction within each rectangular region is a constant.

# Decision tree example



- Left: Suppose (normalised) precipitation  $x_1$  and temperature  $x_2$  determine survival/death ( $y = 1$  or  $0$ ) of a plant species by a rectangular "habitable zone" in the  $x_1 - x_2$  plane (left).
- Right: The survival/death classification can be represented by a decision tree (right).

# Decision tree example



- Classification function for feature vector  $\mathbf{x} = (x_1, x_2)$

$$f(\mathbf{x}; \boldsymbol{\theta}) = \begin{cases} 0 & \text{if } x_1 < 0 \\ 0 & \text{if } x_1 > 0, x_2 > 0.5 \\ 0 & \text{if } x_1 > 0, x_2 < -0.5 \\ 1 & \text{otherwise} \end{cases}$$

- If decision thresholds are trainable parameters, a decision tree becomes a ML model for classification.

# Decision tree terminology

- **Split:** Partitioning data based on a feature and threshold
- **Node:** a point in the tree where a decision is made
- **Root node:** first split in the tree
- **Internal node:** A node that performs a split
- **Leaf node:** Final node that outputs a prediction
- **Depth:** Number of decision levels from Root node to leaves
- **Impurity:** Measure how mixed the targets are within a split
- **Information gain:** Reduction in impurity after a split

# Training a decision tree for binary classification

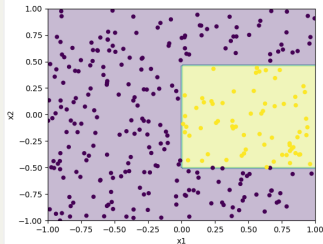
```
from sklearn.tree import DecisionTreeClassifier

# synthetic data
# X.shape = (300, 2) -- x1,x2 as columns
# y.shape = (300,) -- targets

# initialise decision tree with max. depth 3
clf = DecisionTreeClassifier(max_depth=3)

# train decision tree
clf.fit(X, y)

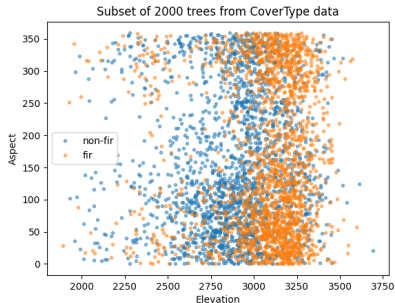
# predict
clf.predict(np.array([0.5,0]).reshape(-1,2))
# returns: array([1])
```



The trained decision tree recovers the correct classification boundary in this simple example.

# Code example: Tree cover classification

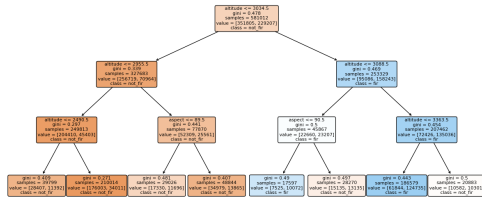
```
import gzip
import numpy as np
path = "covtype.data.gz"
with gzip.open(path, "rt") as f:
    arr = np.genfromtxt(f, delimiter=",")
```



- Cover type data contains data from over 500,000 trees, including topographic, environmental, hydrological, and species information
- Here we wish to predict whether a tree is a pine species, given its height (metres) and aspect (degrees from North)

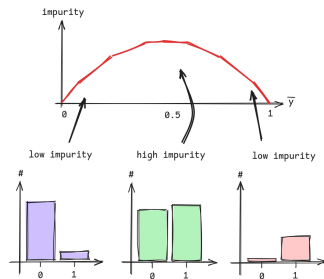
# Visualising trained decision trees

```
from sklearn import tree
clf = DecisionTreeClassifier(
    max_depth=3)
clf.fit(X, y)
plt.figure(figsize=(10, 6))
tree.plot_tree(clf,
    feature_names=["altitude",
                   "aspect"],
    class_names=["not_fir",
                 "fir"],
    filled=True, rounded=True,
    fontsize=8)
plt.show()
```



# Finding features and thresholds for splitting

- Gini impurity and entropy are measures of impurity, disorder, indeterminism.
- Binary data:  
 $\mathbf{y} = (y_1, \dots, y_n) = (0, 0, 1, 0, 1, \dots)$ .
- Summarise  $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ .



- Gini impurity:  $G(\mathbf{y}) = 1 - \bar{y}^2 - (1 - \bar{y})^2$
- Entropy:  $H(\mathbf{y}) = \bar{y} \log \bar{y} + (1 - \bar{y}) \log(1 - \bar{y})$



# Maximising Information Gain

- We apply threshold-splitting of the data wrt one feature.
- This separates the data  $\mathbf{y}$  into  $\mathbf{y}_{left}$  and  $\mathbf{y}_{right}$  of sizes  $n_{left}$  and  $n_{right}$ .
- The impurity of the split data set is calculated by

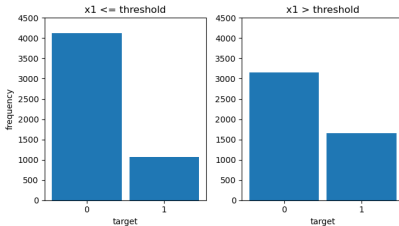
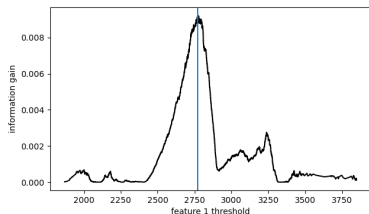
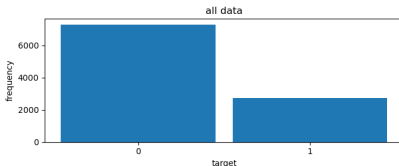
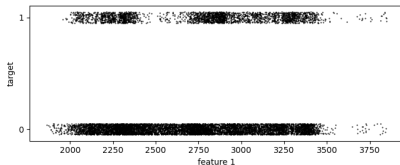
$$G(\mathbf{y}_{left}, \mathbf{y}_{right}) = \frac{n_{left}}{n} G(\mathbf{y}_{left}) + \frac{n_{right}}{n} G(\mathbf{y}_{right})$$

- The impurity reduction (or "information gain"  $IG$ ) caused by the split is given by

$$IG = G(\mathbf{y}) - G(\mathbf{y}_{left}, \mathbf{y}_{right})$$

- The feature/threshold combination that yields the highest information gain is selected.

# Maximising Information Gain



# Feature importance

- To interpret a trained ML model it is often interesting to ask "which feature contributes most to the model skill?"
- This is a very difficult question when there are complex interactions between features.
- Some options include
  - Quick and dirty, for decision trees:  
`clf.feature_importances_` (mean impurity decrease)
  - More informative and generic:  
`sklearn.inspection.permutation_importance`  
(performance drop when a feature is randomly shuffled)
  - Widely used, but complicated, measure for reporting: SHAP values (global importance measure)
  - Many more (further reading)

# Hyperparameters

- A hyperparameter is a model parameter that is not inferred from the training data by the learning algorithm.
- For decision trees, model parameters are the decision thresholds, and hyperparameters can be
  - minimum/maximum tree depth
  - maximum number of leaf nodes
  - minimum number of samples per split
  - impurity measure



# Hyperparameter optimisation

- Cross-validation:
  - randomly split the training data into  $n_{CV}$  subsets (usually 5 or 10)
  - use  $n_{CV} - 1$  subsets to train the model
  - evaluate the model on the left-out subset
  - repeat for each of the  $n_{CV}$  folds
  - average the  $n_{CV}$  out-of-sample losses into a single cross-validation loss
- Grid search:
  - Enumerate all  $n_{comb}$  combinations of hyperparameters
  - Calculate cross validation loss for each combination
  - Choose the hyperparameter setting with lowest cv loss
  - Fit the model with the optimised hyperparameters
- The computational cost of exhaustive grid search with cross validation is  $n_{CV} \times n_{comb}$  larger than training a single model.

# Hyperparameter optimisation

```
from sklearn.model_selection import GridSearchCV

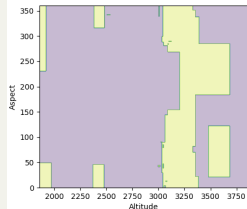
# see all model hyperparameters
clf.get_params()

# initialise grid search
clf_cv = GridSearchCV(
    estimator=clf,
    param_grid={"max_depth": [4,8,16,32],
                "criterion": ["gini", "entropy"]},
    cv=5
)

# run grid search
clf_cv.fit(X, y)

# see best hyperparameters
clf_cv.best_params_

# predict using optimised model
clf_cv.predict(X)
```



# Random Forest

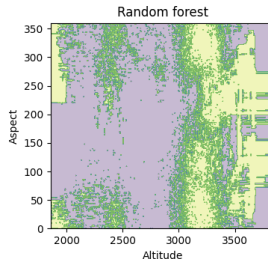
- A RF is an ensemble of indepently trained decision trees.
- Each tree is trained on random subsets of training data and input features.
- Each tree makes a prediction and the forest aggregates them by:
  - Majority vote (Classification)
  - Averaging (Regression)
- Ensembling improves robustness and avoids overfitting compared to single decision tree.

# Code example: Random forest

```
from sklearn.ensemble import RandomForestClassifier

# initialise random forest classifier
rfc = RandomForestClassifier(
    n_estimators=100, # number of trees
    max_depth=20,     # maximum tree depth
    random_state=42    # reproducibility
)

# train and predict
model.fit(X, y)
y_pred = model.predict(X)
```





# Boosting

- Ensemble methods (such as RF) train a number of **strong** classifiers **independently**.
- **Boosting methods** trains a chain of **weak** classifiers **sequentially**.
- A "weak learner" is simple, fast to train, and has low individual accuracy.
- Core algorithm:
  - train a simple model
  - calculate the model's errors over the training data
  - train the next model to take the same inputs but predict the previous model's errors
  - repeat, then sum the outputs of all models (often with weights)
- Throughout the boosting sequence the model is improved by gradually refining the prediction.

# Gradient Boosting

Technical details of gradient boosting:

- **Gradients** of the loss function  $g_i = \frac{\partial L(\hat{y}_i, y_i)}{\partial \hat{y}_i}$  approximate prediction errors/residuals.
- Changing  $\hat{y}_i$  proportional to  $-g_i$  will decrease the loss.
- In **gradient boosting**, each tree predicts the negative gradient  $\frac{\partial L(\hat{y}, y)}{\partial \hat{y}}$  of the loss of the previous tree.
- If  $f_{i+1}(\mathbf{x}; \theta_{i+1})$  is the output of the  $i + 1$ -st learner, the  $i + 1$ st model prediction is

$$\hat{y}_{i+1} = \hat{y}_i + \eta f_{i+1}(\mathbf{x}; \theta_{i+1})$$

where  $\eta$  is the **learning rate**.

# XGBoost



- XGBoost is a popular and efficient library for Gradient Boosting models for regression and classification
- Website: `xgboost.ai`
- Very similar naming conventions to scikit-learn:

```
import xgboost as xgb
clf = xgb.XGBClassifier(n_estimators = 100, max_depth=5)
clf.fit(X_train, y_train)
y_hat = clf.predict(X_test)
```

# XGBoost for Boosted Decision Trees

```
import xgboost as xgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# simple XGB model: very weak learners, few boosting rounds
clf_xgb = xgb.XGBClassifier(
    n_estimators = 10,
    max_depth = 1,
    random_state = 42
)

# fit and predict
clf_xgb.fit(X_train, y_train)
y_pred_xgb = clf_xgb.predict(X_test)

# evaluate test data
accuracy_score(y_test, y_pred_xgb)
# accuracy about 0.72

# random forest with same complexity
clf_rf = RandomForestClassifier(
    n_estimators=10, random_state=42, max_depth=1)
clf_rf.fit(X_train, y_train)
y_pred_rf = clf_rf.predict(X_test)
accuracy_score(y_test, y_pred_rf)
# accuracy only about 0.6
```

# Evaluation metrics in scikit-learn

- Scikit-learn offers numerous evaluation metrics to compare a set of predictions  $\hat{y}_1, \dots, \hat{y}_n$  to a corresponding targets  $y_1, \dots, y_n$ .
- Some commonly used evaluation metrics/methods are
  - Accuracy
  - Confusion matrix
  - Classification report
  - ROC/AUC analysis
- See [scikit-learn.org/stable/modules/model\\_evaluation.html](https://scikit-learn.org/stable/modules/model_evaluation.html) for a full list and details.

## Introduction

- Supervised ML
- Model training
- Examples, model evaluation

## Tree-based methods

- Decision trees
- Random Forest, XGBoost

## Neural Networks

# Artificial neuron (Perceptron)

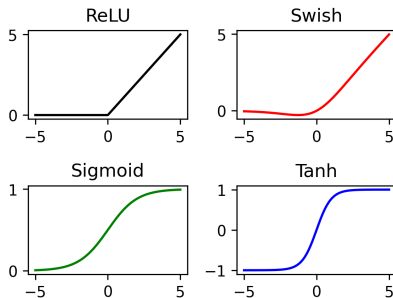
- An artificial neuron (perceptron) is a mathematical function

$$\hat{y} = f(\mathbf{x}; \mathbf{w}, b) = \varphi(w_1 \cdot x_1 + \dots + w_k \cdot x_k + b)$$

- where
  - $\mathbf{x} = (x_1, \dots, x_k)$  is the  $k$ -dimensional input vector
  - $y$  is the 1-dimensional output of the neuron
  - $\mathbf{w} = (w_1, \dots, w_k)$  is the  $k$ -dimensional vector of weights
  - $b$  is the bias
  - $\varphi(\cdot)$  is the activation function
- A neuron takes inputs  $\mathbf{x} = (x_1, \dots, x_k)$ , calculates a **linear combination**, and applies a non-linear activation function.

# Activation functions

- Typical activation functions:
  - ReLU (Rectifier):  $\varphi(x) = \max(0, x)$
  - Sigmoid:  $\varphi(x) = 1/(1 + e^{-x})$
  - Swish:  $\varphi(x) = x/(1 + e^{-x})$
  - Tanh:  $\varphi(x) = (e^x - e^{-x})/(e^x + e^{-x})$

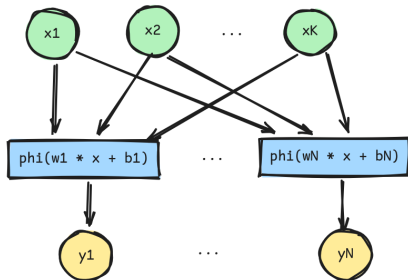




# Single-layer perceptron

- In a SLP, **several perceptrons** (each with different weights and biases) calculating several outputs in parallel
- produce a **vector of outputs**  $y_1, \dots, y_N$  where

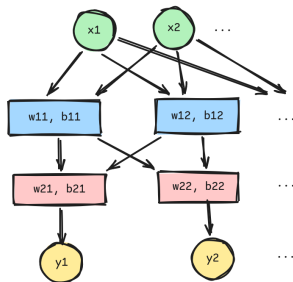
$$y_i = \varphi(w_{i,1}x_1 + \dots + w_{i,k}x_k + b_i) = \varphi(\mathbf{w}_i \cdot \mathbf{x} + b_i)$$



# Multilayer perceptron

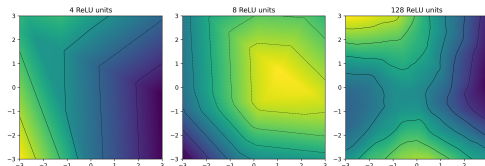
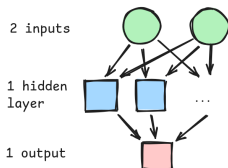
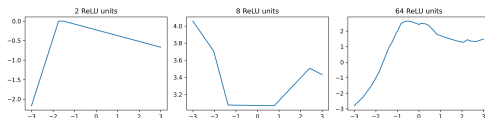
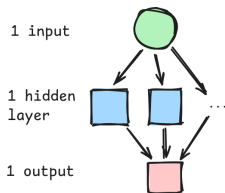
- In an MLP, **perceptrons are arranged in multiple layers**
- Each layer computes a **vector of activations** that serves as input to the next layer
- For layer  $\ell$ , the vector of outputs  $\mathbf{y}^{(\ell)} = (y_1^{(\ell)}, \dots, y_{n_\ell}^{(\ell)})$  is calculated from the previous one by

$$y_i^{(\ell)} = \varphi \left( \mathbf{w}_i^{(\ell)} \cdot \mathbf{y}^{(\ell-1)} + b_i^{(\ell)} \right)$$



# Examples

Randomly initialised ReLU networks with one hidden layer:



More nodes (units) produce more "flexible" curves.

# Terminology

- **Weight:** trainable parameter scaling an input
- **Bias:** trainable offset added to the weighted sum
- **Pre-activation:** linear combination  $z = w \cdot x + b$
- **Activation function:** nonlinear function applied to pre-activation
- **Input layer:** layer providing the input features
- **Output layer:** final layer producing the model output(s)
- **Hidden layer(s):** layer(s) between input and output layers
- **Parameters:** all trainable weights and biases of the network
- **Forward pass:** computation of outputs from inputs by the network

# PyTorch, TensorFlow

- Low-level frameworks for implementing neural networks.
- Automatic differentiation, backpropagation.
- Fast training on multicore CPUs and GPUs "out of the box".
- PyTorch is used in research, TF more widely used in production.
- See [pytorch.org](https://pytorch.org) and [tensorflow.org](https://tensorflow.org).

# Keras

- High-level API for building NNs, based on TensorFlow or PyTorch.
- Simple syntax for composing standard layers into complex deep NN architectures.
- Ideal for learning and prototyping.
- See [keras.io](https://keras.io).

# Simple data example

# Model summary



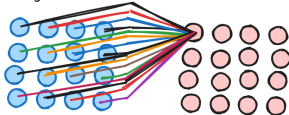
# Loss functions

crossentropy-loss for classification, squared-error loss for regression

# Convolution

- Perceptrons are **densely connected** layers because every node in a layer receives outputs from all nodes in the previous layer.
- When inputs and outputs are high-dimensional, the number of trainable parameters in a dense layer becomes very large.
- Convolution is a sparsely connected layer with where outputs are only connected to **spatially close inputs** and weights are **shared across the domain**.

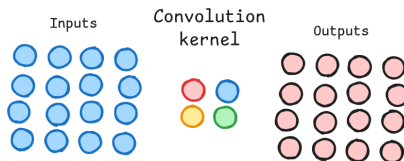
Densely connected with individual weights



Locally connected with shared weights



# Convolution



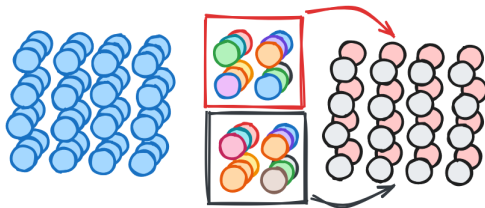
# Convolution

- The input data usually have multiple channels (color channels, pressure levels, variables)
- The convolution kernel must have the same number of channels as the input data.



# Convolution

- By defining multiple convolution kernels, we produce multiple output channels from the same input.



- This is the typical setup for a 2d convolution layer in a convolutional neural network.
- The trainable parameters in a 2d convolution are stored in a 4d data structure with dimensions (width, height, input channels, output channels).

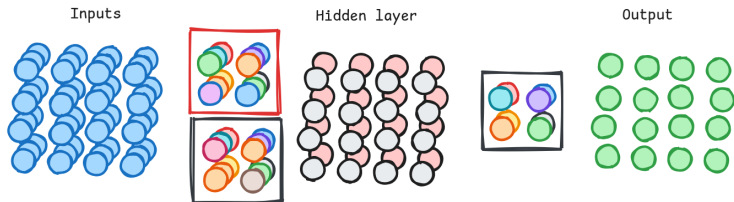
# Convolution

- inputs  $x[i, j, c]$  indexed by space  $(i, j)$  and channel  $c = 1, \dots, C$
- outputs  $y[i, j]$  indexed by space  $(i, j)$
- kernel  $k[m, n, c]$  of width and height  $m = -w, \dots, +w$  and  $C$  channels

$$y[i, j] = \sum_{m=-w}^w \sum_{n=-w}^w \sum_{c=1}^C k[m, n, c] \cdot x[i + m, j + n, c]$$

# Convolution

- Convolution layers can be chained to produce a deep convolutional neural network (Deep CNN).



# Typical setup

- A common situation in ML with spatial data:
  - several input variables, each a 2d spatial field (input channels)
  - the target is a single spatial field of the same size (output channel)
- A typical CNN for such data has:
  - 2-6 hidden convolution layers
  - kernel size 3 or 5
  - 16-256 channels per hidden layer
  - 1 channel at output layer (to match the target)
- This will produce models with  $10^4$  -  $10^6$  parameters, suitable for training locally on moderately large training data sets
- These hyperparameters can be explored and optimised as part of the ML model development.



# Examples

- Hazard prediction: MORE HERE
- Image segmentation: MORE HERE
- Next-time step prediction: MORE HERE

## Worked example: Radar nowcasting

MORE HERE

# Gradient descent

# Backpropagation

# Stochastic gradient descent

# Optimiser hyperparameters

# Monitoring performance

learning curves, restarting training runs

# Encoder-Decoder architecture



# Example: Weather image classification

# Cross-validation, early stopping

# Dropout

## Further Reading

- Sensitivity analysis, interpretability
- Fine-tuning for new outputs/objectives
- Speeding up NNs: Quantisation, layer-wise optimisation
- MORE HERE